

Canlı Veri Akışı Üzerinde İçerik Tabanlı Dağıtık Öneri Sistemi

Proje Ara Raporu

Arda Onat Acar 231401010
Nihat Emre Yüzügöldü 221401009

13 Temmuz 2026

1. Projenin Kısa Özeti

Bu projede amaç, takip grafi kullanmadan, kullanıcının belirttiği ilgi alanları ile sisteme gelen gönderiler arasındaki anlamsal benzerliği ölçerek kişiselleştirilmiş bir akış üretmektir. İlk aşamada Kafka, Spark, hazır bir metin embedding modeli, Qdrant ve Scala tabanlı bir servis katmanının birlikte kullanılması planlanmaktadır.

İlk prototipte yeni bir yapay zekâ modeli eğitilmemektedir. Bunun yerine, hazır bir çok dilli embedding modeli kullanılarak kullanıcı ilgi alanları ve gönderiler sayısal vektörlere çevrilmiştir. Benzer vektörler birbirine yakın kabul edilmekte ve en ilgili gönderiler kullanıcıya önerilmektedir.

Şimdiye kadar yapılan çalışmalar çoğunlukla hazır ve açık kaynak bileşenlerin incelenmesi ile bu bileşenlerin birlikte nasıl kullanılacağına planlanmasından oluşmaktadır.

2. Problem Tanımı ve Kapsam

Sosyal medya uygulamalarında kullanıcıya gösterilecek içerik sayısı çok fazladır. İlk prototip, bu içeriklerin tamamını karmaşık sosyal grafiklerle değerlendirmek yerine daha dar bir problemi çözmektedir:

- Kullanıcının ilgi alanlarını metin olarak almak.
- Yeni gönderileri sürekli bir veri akışı halinde sisteme almak.
- Gönderilerin anlamını hazır bir modelle sayısal vektöre çevirmek.
- Kullanıcının ilgi vektörüne en yakın gönderileri bulmak.
- Benzerlik skoruna ek olarak gönderinin güncelliğini de hesaba katmak.

Bu aşamada kapsam dışında bırakılanlar: takip grafi, kullanıcılar arası collaborative filtering, özel model eğitimi, gelişmiş tıklama tahmini, reklam sistemi, güvenlik/moderasyon kuralları ve üretim seviyesinde hata toleransı.

3. Bileşenler ve Kodun Basit Açıklaması

3.1. Veri Üretici (Producer)

Veri üretici, sisteme gönderi sağlayacak en basit modül olarak planlanmıştır. İlk aşamada bir JSONL dosyasındaki örnek gönderilerin belirli aralıklarla Kafka'ya iletilmesi düşünülmektedir. Daha sonra aynı yapı canlı bir sosyal medya akışına bağlanabilir.

```
{
  "postId": "post-101",
  "authorId": "author-8",
  "text": "Spark Structured Streaming ile gerçek zamanlı veri işleme",
  "language": "tr",
  "createdAt": "2026-07-13T10:00:00Z"
}
```

3.2. Apache Kafka

Kafka, servisler arasındaki mesajlaşma katmanıdır. Veri üretici gönderiyi doğrudan Spark veya embedding servisine göndermez; önce Kafka topic'ine yazar. Böylece bir servis geçici olarak kapansa bile veriler kaybolmadan daha sonra işlenebilir.

- posts.raw: Sisteme ilk gelen ham gönderiler.
- posts.cleaned: Spark tarafından kontrol edilmiş temiz gönderiler.

3.3. Spark Structured Streaming

Scala ile hazırlanacak Spark işinin posts.raw topic'ini dinlemesi, bozuk veya boş kayıtları ayıklaması ve aynı postId ile gelen tekrar kayıtları kaldırması planlanmaktadır. Kontrolten geçen kayıtlar posts.cleaned topic'ine aktarılacaktır.

```
Kafka posts.raw
-> JSON parse et
-> zorunlu alanları kontrol et
-> postId tekrarlarını kaldır
-> Kafka posts.cleaned
```

Spark burada öneri kararı vermemektedir. Görevi yalnızca veri akışını güvenilir ve düzenli hale getirmektir. İleride birden fazla makineye geçildiğinde aynı iş Spark executor'ları arasında dağıtılabilir.

3.4. Embedding Worker

Embedding worker, temiz gönderileri Kafka'dan okuyacak küçük bir Python servisi olarak planlanmıştır. Hazır çok dilli metin modeliyle her gönderi, anlamını yaklaşık olarak temsil eden sabit uzunlukta bir sayı dizisine dönüştürülecektir.

```
"CUDA kernel optimizasyonu" -> [0.12, -0.08, 0.41, ...]
"GPU programlama teknikleri" -> [0.11, -0.06, 0.39, ...]
```

Anlamları yakın olan iki metnin vektörleri de birbirine yakın olur. Böylece sadece aynı kelimeleri içeren metinler değil, benzer anlam taşıyan metinler de eşleştirilebilir.

3.5. Qdrant Vektör Veritabanı

Qdrant, gönderi vektörlerini ve gönderiye ait metaverileri saklar. Normal bir veritabanında postId ile arama yapılırken, Qdrant içinde "bu vektöre en yakın 100 vektörü getir" sorgusu yapılabilir. Öneri sisteminin temel arama işlemi budur.

3.6. Scala Recommendation API

Kullanıcının ilgi alanlarını alacak ve öneri listesini döndürecek servisin Scala ile hazırlanması planlanmaktadır. Servisin iki temel işlemi olacaktır:

1. Kullanıcının ilgi alanlarını embedding servisine gönderip tek bir kullanıcı profili vektörü oluşturmak.
2. Bu vektörle Qdrant üzerinde benzer gönderileri aramak ve sonuçları basit bir formülle yeniden sıralamak.

```
POST /users/user-1/interests
{
  "interests": ["Scala", "dağıtık sistemler", "GPU programlama"]
}

GET /users/user-1/feed?limit=20
```

3.7. Docker Compose ve Kubernetes

İlk geliştirme ortamında servislerin Docker Compose ile tek makinede başlatılması planlanmaktadır. Böylece bileşenlerin birlikte çalışması daha kolay test edilebilecektir. Daha sonraki aşamada servisler Kubernetes deployment'larına dönüştürülebilir.

Servislerin birbirinden ayrı tutulması, ileride Kafka partition, Spark executor ve servis replica sayılarının artırılmasına imkân verecektir.

4. Planlanan Uçtan Uca Çalışma Akışı

1. Örnek veya canlı bir gönderi veri üretici tarafından posts.raw topic'ine yazılır.

2. Spark gönderiyi okur, JSON formatını kontrol eder ve tekrar kayıtları temizler.
3. Temiz gönderi posts.cleaned topic'ine yazılır.
4. Embedding worker gönderiyi okur ve hazır modelle vektöre dönüştürür.
5. Vektör, gönderi metni ve tarih bilgisiyle birlikte Qdrant'a kaydedilir.
6. Kullanıcı ilgi alanlarını Scala API'ye gönderir.
7. İlgili alanları da vektöre dönüştürülür ve ortalaması kullanıcı profili olarak saklanır.
8. Feed isteği geldiğinde API, kullanıcı vektörüne en yakın gönderileri Qdrant'tan alır.
9. Sonuçlar benzerlik ve güncellik skoruna göre sıralanarak kullanıcıya döndürülür.

4.1. Basit Sıralama Formülü

İlk prototipte ayrıca eğitilmiş bir ranking modeli kullanılmaması planlanmaktadır. Bunun yerine Qdrant'ın verdiği anlamsal benzerlik skoru ile gönderinin güncelliği birleştirilecektir:

$$\text{finalScore} = 0.85 * \text{semanticSimilarity} + 0.15 * \text{recencyScore}$$

Bu formül bilinçli olarak basit tutulmuştur. Amaç, ilk aşamada model kalitesini en üst düzeye çıkarmak değil, veri akışının temel bileşenlerini küçük bir prototip üzerinde denemektir.

6. Planlanan Dağıtık Sistem Özellikleri

Öneri algoritması basit tutulurken veri işleme hattının ileride dağıtılabilecek biçimde kurulması hedeflenmektedir. Planlanan dağıtık özellikler şunlardır:

- Kafka topic'leri partition'lara ayrılarak birden fazla consumer tarafından paralel okunabilir.
- Spark işi birden fazla executor üzerinde farklı veri parçalarını paralel işleyebilir.
- Birden fazla embedding worker aynı consumer group içinde çalışabilir.
- Recommendation API birden fazla replica ile yatay ölçeklenebilir.
- Servisler Kubernetes üzerinde farklı makinelerde bağımsız pod'lar olarak çalıştırılabilir.

Bu yaklaşımda verinin taşınması, temizlenmesi, vektöre dönüştürülmesi, saklanması ve servis edilmesi gibi aşamaların ileride ayrı ayrı ölçeklenebilmesi amaçlanmaktadır.

7. Ara Rapor Aşamasındaki Durum ve Sınırlamalar

Ara rapor aşamasında hedef, sistemin baştan sona çalışan en küçük sürümünü oluşturmaktır. Bu nedenle bazı kararlar özellikle basit tutulmuştur.

- Kullanıcı ilgisi, davranışlardan otomatik öğrenilmek yerine kullanıcı tarafından açıkça girilmektedir.
- Takip grafi ve sosyal bağlantılar dikkate alınmamaktadır.
- Gönderi kalitesi, güvenilirliği ve güvenlik kontrolleri henüz değerlendirilmemektedir.
- Ranking için özel eğitim yapılmamaktadır; hazır embedding benzerliği kullanılmaktadır.
- İlk testler sınırlı veri ve tek makine üzerinde yapılabilir; çok makineli test sonraki aşamadır.
- Gerçek zamanlılık, anlık değil birkaç saniyelik gecikme toleransı ile ele alınmaktadır.

8. Sonraki Aşamada Yapılabilecek Geliştirmeler

- Kullanıcının tıklama, beğeni ve görüntüleme süresinden ilgi profilini otomatik güncellemek.
- Benzerlik yanında popülerlik, kalite, dil ve yazar çeşitliliği özellikleri eklemek.
- Basit formül yerine eğitilmiş bir ranking modeli kullanmak.
- Aynı gönderiyi tekrar göstermemek için impression geçmişi tutmak.
- Kubernetes üzerinde birden fazla replica ve hata senaryosu test etmek.

- Canlı sosyal medya kaynağını doğrudan producer modülüne bağlamak.

9. Sonuç

Bu ara raporda, canlı akan gönderileri kullanıcının ilgi alanlarına göre sıralamayı amaçlayan basit bir içerik tabanlı öneri sistemi tasarımı açıklanmıştır. Çalışmanın odağı, Kafka, Spark, hazır embedding modeli, vektör veritabanı ve Scala servis katmanının aynı veri hattında nasıl bir araya getirilebileceğini incelemektir.

İlk prototipin küçük bir kapsamda tutulması planlanmaktadır. Daha sonraki aşamalarda yeni veri kaynakları, modeller ve sıralama yöntemleri eklenebilir.